

Document number:	P0184R0
Date:	2016-02-11
Project:	Programming Language C++, Evolution Working Group
Reply-to:	Eric Niebler < eniebler@boost.org >

Generalizing the Range-Based For Loop

“In the end, we are self-perceiving, self-inventing, locked-in mirages that are little miracles of self-reference.”

– Douglas Hofstadter, *I Am a [St]range Loop*

- [1 Introduction](#)
- [2 Motivation and Scope](#)
- [3 Implementation Experience](#)
- [4 Technical Specifications](#)
- [5 Acknowledgements](#)
- [References](#)

1 Introduction

The current draft of the [Ranges TS](#)[2] loosens the requirements on the objects used to denote a range. Today, two iterators of the same type are used to denote a range. In the Ranges TS, a *sentinel* may be used to denote the end of a range, where the type of the sentinel may be different from the type of the range’s iterator. The motivation for the loosening of this requirement is given in section 3.3.5 of [Ranges for the Standard Library, Revision 1](#)[3], but in short it is to facilitate better code generation for many interesting kinds of ranges, such as a range denoted by an iterator and a predicate.

Allowing the type of a range’s end to differ from that of its begin causes problems when the user tries to use such a range with the built-in range-based `for` loop, which requires a range’s begin and end to have the same type. This paper proposes to lift that restriction for C++17, thereby giving users of the Ranges TS the best possible experience.

2 Motivation and Scope

The existing range-based `for` loop is over-constrained. The end iterator is never incremented, decremented, or dereferenced. Requiring it to be an iterator serves no practical purpose.

Today, STL algorithms require the begin and end of a range to have the same type. That is sensible given the fact that algorithms take the begin and end of a range as separate parameters. In the absence of concept checking, allowing their types to differ makes it rather easy to pass mismatched iterators. The range-based `for` loop has no such problem since it deals with ranges in whole.

Loosening the type requirements of the range-based `for` loop gives users of the Ranges TS the best possible experience. If users create ranges with the Ranges TS are not usable with the built-in range-based `for` loop, they will be frustrated, and it will reflect poorly both on the Ranges TS and on the range-based `for` loop. In all likelihood, a macro-based solution like [BOOST_FOREACH](#)[1] will be invented to fill the gap. That is best avoided.

3 Implementation Experience

The author has implemented the described resolution in the clang compiler, and Casey Carter has implemented it in gcc. For clang, the implementation was as simple as removing the code that checks that `begin()` and `end()` return objects of the same type. The change for gcc was equally trivial. After this change was made, non-bounded ranges (those for which `end()` returns a sentinel that is not an iterator) work with the built-in range-based `for` loop.

4 Technical Specifications

6.5.4 The range-based for statement [stmt.ranges]

1. A range-based for statement is equivalent to

```
{
auto && __range = for-range-initializer;
for ( auto __begin = begin-expr,
      __end = end-expr;
      __begin != __end;
      ++__begin ) {
    for-range-declaration = *__begin;
    statement
}

{
    auto && __range = for-range-initializer;
    auto __begin = begin-expr;
    auto __end = end-expr;
    for ( ; __begin != __end; ++ __begin ) {
        for-range-declaration = * __begin;
        statement
    }
}
```

where [...] [*Editorial note: – ... as in the current Working Draft. –end note*]

5 Acknowledgements

I would also like to thank Herb Sutter and the Standard C++ Foundation, without whose generous financial support the Ranges TS would not be possible.

References

- [1] Boost.Foreach: <http://boost.org/libs/foreach>. Accessed: 2015-11-18.
- [2] Niebler, E. and Carter, C. 2015. N4560: Working Draft, C++ Extensions for Ranges.
- [3] Niebler, E. et al. 2014. N4128: Ranges for the Standard Library, Revision 1.